

---

## Nirma University — Institute of Technology

---

Department of Electronics & Communication Engineering

**ELECTRONICS CIRCUITS (2EC105CC25)**

Project Report

# Bluetooth Control Car

Implementation on Arduino UNO (ATmega328P)

---

**Submitted By:**

Harshiul Manasara (24BEC139), Arjun Hirani (24BEC138)

**Guided By:**

**Academic Year: 2025–2026**

Semester: Even

# 1. Introduction

---

A Bluetooth Controlled Car is a simple robotic system that can be operated wirelessly using a mobile phone through Bluetooth communication. It is commonly used in basic robotics projects to understand how wireless control and embedded systems work together. In this project, an Arduino UNO microcontroller acts as the main control unit, while a Bluetooth module such as HC-05 enables communication between the car and the smartphone.

The working of the system is very simple. The user sends commands like forward, backward, left, right, and stop through a mobile application. These commands are transmitted via Bluetooth to the Arduino. The Arduino reads these signals and processes them, then sends control instructions to the motor driver (L298N), which drives the DC motors connected to the wheels.

The motor driver plays an important role because the Arduino cannot supply enough current to run the motors directly. Therefore, the motors are connected to the motor driver, which is powered by a battery. The Arduino only gives control signals, while the motor driver provides the required power to rotate the motors and move the car.

The main objective of this project is to understand how Bluetooth communication is used to control a vehicle, learn the function of each component in the system, and observe the real-time response of a mobile-controlled robotic car. This project also helps in building a strong foundation in robotics and automation.

## 2. Component List

---

The following hardware and software tools were used:

| Reference | Component                   | Qty | Description / Role                                                      |
|-----------|-----------------------------|-----|-------------------------------------------------------------------------|
| U1        | Arduino UNO R3 (ATmega328P) | 1   | Main microcontroller — processes Bluetooth commands and controls motors |
| U2        | L293D Motor Driver Module   | 1   | Drives DC motors and controls direction/speed using Arduino signals     |
| U3        | HC-05 Bluetooth Module      | 1   | Enables wireless communication between mobile phone and Arduino         |
| M1–M4     | DC Motors (3V–6V)           | 4   | Provides motion to the car wheels                                       |
| –         | Car Chassis with Wheels     | 1   | Mechanical structure of the RC car                                      |
| –         | Lithium Batteries (18650)   | 2   | Power supply for motors and Arduino                                     |
| –         | Battery Holder              | 1   | Holds and connects batteries safely                                     |

Table 1: Hardware and software components used in the bluetooth control car

Note: No extra electronic components like resistors, capacitors, or inductors are required for this project because all control is done through programming in the Arduino. The Arduino's built-in 16 MHz clock helps in processing commands quickly and ensures smooth and accurate operation of the Bluetooth controlled car.

### 3. System Architecture

#### 3.1 Block Diagram of Bluetooth Controlled Car

A Bluetooth controlled car system consists of three main parts: an **Input Unit**, a **Control Unit**, and an **Output Unit**. In this project, each part is implemented using Arduino and its connected components, where the input is received through Bluetooth, processed by the Arduino, and then converted into motor movement.

| System Block           | Arduino Equivalent               | Implementation Detail                                                                |
|------------------------|----------------------------------|--------------------------------------------------------------------------------------|
| Input Command          | Bluetooth Serial (RX/TX Pins)    | Receives commands (F, B, L, R, S) from mobile via HC-05 using UART communication.    |
| Command Processing     | Arduino loop() Logic             | Reads serial data and interprets characters to decide movement direction.            |
| Motor Control Logic    | Digital Output Pins (D2–D7)      | Sends HIGH/LOW signals to motor driver inputs based on received command.             |
| Motor Driver Interface | L293D Motor Driver IC            | Amplifies Arduino signals to drive DC motors and control direction.                  |
| Actuation (Movement)   | DC Motors (Connected via Driver) | Motors rotate in different directions to move car forward, backward, left, or right. |

Table 2: Mapping of PLL sub-blocks to Arduino firmware and hardware

## 3.2 Signal Flow

In this project, the signal flow starts from the **mobile phone**, where the user sends commands like forward, backward, left, right, or stop using a Bluetooth control app. These commands are transmitted wirelessly through the Bluetooth module (HC-05) and received by the Arduino UNO. The Bluetooth module sends this data to the Arduino through serial communication.

After receiving the command, the Arduino processes it inside the program (code). It checks which command is received and decides what action to perform. For example, if the command is "F" (Forward), the Arduino prepares signals to move the car forward. Similarly, different commands result in different movements.

These control signals are then sent to the motor driver (L298N). The motor driver acts as a bridge between the Arduino and the motors. It takes low-power signals from the Arduino and converts them into high-power output suitable for running the DC motors. Based on the signals, it controls the direction and rotation of the motors.

Finally, the motors rotate according to the given command, and the car moves in real time. This completes the signal flow from the mobile phone to the movement of the car, allowing smooth and wireless control of the vehicle.

## 4. Arduino UNO Hardware Description

### 4.1 Microcontroller Overview

The Arduino UNO R3 is the main controller used in this project, based on the ATmega328P microcontroller running at 16 MHz. It acts as the brain of the Bluetooth controlled car, processing commands received from the Bluetooth module and controlling the motors accordingly. It has 14 digital input/output pins (with 6 PWM pins for speed control) and 6 analog input pins. The board can be powered using a USB cable or battery, and it is easy to program using the Arduino IDE, making it suitable for beginners and robotics projects.

### 4.2 Pin Assignment for This Experiment

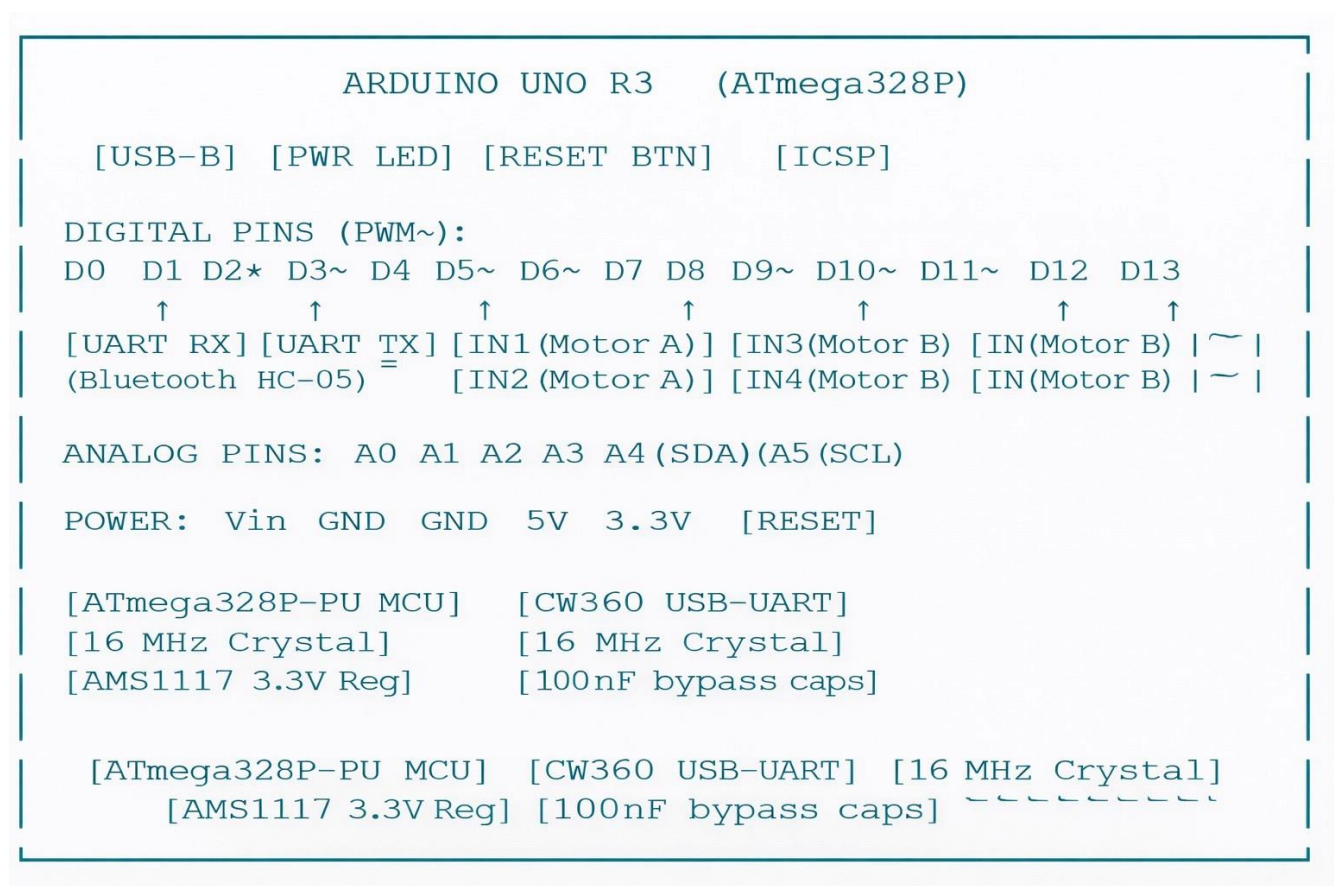


Figure 1: Arduino UNO R3 — Annotated pin diagram for the PLL experiment

| Pin    | Label         | Mode           | Function                                             |
|--------|---------------|----------------|------------------------------------------------------|
| 1 (TX) | UART TX       | Serial Output  | Transmits data to HC-05 module for communication.    |
| 2      | IN1 (Motor A) | Digital Output | Controls direction of Motor A (Forward/Backward).    |
| 3      | IN2 (Motor A) | Digital Output | Controls direction of Motor A (Forward/Backward).    |
| 4      | IN3 (Motor B) | Digital Output | Controls direction of Motor B (Left/Right movement). |
| 5      | IN4 (Motor B) | Digital Output | Controls direction of Motor B (Left/Right movement). |
| 6(RX)  | UART RX       | Serial Input   | Receives Bluetooth data from HC-05 module.           |

Table 3: Pin assignments

### 4.3 Timer 1 Configuration

In this project, the Arduino does not use Timer 1 in CTC mode like a frequency generator, but it uses its internal clock and PWM pins to control the motors. The Arduino runs at 16 MHz, which allows it to process Bluetooth commands quickly and control motor movement smoothly. By using PWM pins, the Arduino can vary the speed of the motors, providing better control over the car's movement. This speed range is sufficient for normal operation of the Bluetooth controlled car and ensures stable and responsive performance.

### 4.4 Interrupt Latency

The ATmega328P microcontroller in the Arduino UNO works very fast and can respond quickly to signals from the Bluetooth module. When a command is received, the Arduino processes it with a very small delay, which is usually not noticeable in this project. Any minor delay in receiving or processing signals does not affect the overall performance because the system continuously updates and responds to new commands. This ensures smooth and real-time control of the Bluetooth controlled car.



## 5. Code Explanation

---

### 5.1 Global Initialization and Serial Communication Setup

```
#include <AFMotor.h>
#include <SoftwareSerial.h>

SoftwareSerial bluetoothSerial(9, 10); // RX, TX

// initial motors pin
AF_DCMotor motor1(1, MOTOR12_1KHZ);
AF_DCMotor motor2(2, MOTOR12_1KHZ);
AF_DCMotor motor3(3, MOTOR34_1KHZ);
AF_DCMotor motor4(4, MOTOR34_1KHZ);

char command;
```

This part of the code is used to set up the basic requirements for the Bluetooth controlled car. It includes important libraries needed for motor control and Bluetooth communication. The SoftwareSerial is used to create communication on pins 9 and 10, which connect to the Bluetooth module for receiving commands from the mobile phone.

The code also initializes four DC motors using the motor driver library. Each motor is assigned a specific channel, allowing the Arduino to control them individually for different movements like forward, backward, and turning.

Finally, a variable is declared to store the incoming command from the Bluetooth device. This setup is essential because it prepares the system to receive instructions and control the motors accordingly, forming the foundation of the entire project.

## 5.2 Setup Function

```
void setup()
{
  bluetoothSerial.begin(9600);
}

void loop() {
  if (bluetoothSerial.available() > 0) {
    command = bluetoothSerial.read();

    stop();

    switch (command) {
      case 'F':
        forward();
        break;
      case 'B':
        back();
        break;
      case 'L':
        left();
        break;
      case 'R':
        right();
        break;
    }
  }
}
```

This code is responsible for controlling the movement of the car using Bluetooth commands sent from a mobile phone. The Arduino first establishes communication with the Bluetooth module, allowing it to receive signals wirelessly. Inside the main loop, it continuously checks whether any command is received and reads it.

Based on the received command, the Arduino decides which direction the car should move. It uses a switch-case structure to match each command with a specific movement like forward, backward, left, or right. Before executing any new movement, the system ensures better control by stopping the previous action.

## 5.3 Forward

```
void forward()
{
    motor1.setSpeed(255);
    motor1.run(FORWARD);
    motor2.setSpeed(255);
    motor2.run(FORWARD);
    motor3.setSpeed(255);
    motor3.run(FORWARD);
    motor4.setSpeed(255);
    motor4.run(FORWARD);
}
```

This function is responsible for moving the car in the forward direction. It controls all four motors connected to the motor driver. First, the speed of each motor is set to 255, which means maximum speed using PWM (Pulse Width Modulation).

After setting the speed, each motor is instructed to rotate in the forward direction. Since all motors are running forward at the same speed, the wheels rotate together, and the car moves straight ahead without turning.

This function is very important because it ensures smooth and stable forward movement. By controlling both speed and direction of all motors, the car can move efficiently in real time when the forward command is received from the Bluetooth device

## 5.4 Back

```
void back()
{
    motor1.setSpeed(255);
    motor1.run(BACKWARD);
    motor2.setSpeed(255);
    motor2.run(BACKWARD);
    motor3.setSpeed(255);
    motor3.run(BACKWARD);
    motor4.setSpeed(255);
    motor4.run(BACKWARD);
}
```

This function is used to move the car in the backward direction. It controls all four motors by first setting their speed to 255, which means maximum speed using PWM. This ensures that the car moves with full power.

After setting the speed, all motors are commanded to rotate in the backward direction. Since all wheels rotate in reverse at the same speed, the car moves straight backward smoothly.

This function is important for controlling reverse movement of the car. It helps the user easily change direction and move the car backward when a command is received from the Bluetooth device.

## 5.5 Left

```
//  FIXED LEFT
void left()
{
    motor1.setSpeed(255);
    motor1.run(FORWARD);
    motor2.setSpeed(255);
    motor2.run(FORWARD);
    motor3.setSpeed(255);
    motor3.run(BACKWARD);
    motor4.setSpeed(255);
    motor4.run(BACKWARD);
}
```

This function is used to turn the car towards the left direction. In this function, the motors on one side of the car rotate forward while the motors on the other side rotate backward. Specifically, motor1 and motor2 move forward, while motor3 and motor4 move backward, all at maximum speed (255). This opposite rotation of wheels causes the car to rotate or turn left smoothly in place or with a sharp turn. This function helps in changing the direction of the car effectively when a left command is received from the Bluetooth device.

## 5.6 Right

```
//  FIXED RIGHT
void right()
{
    motor1.setSpeed(255);
    motor1.run(BACKWARD);
    motor2.setSpeed(255);
    motor2.run(BACKWARD);
    motor3.setSpeed(255);
    motor3.run(FORWARD);
    motor4.setSpeed(255);
    motor4.run(FORWARD);
}
```

This function is used to turn the car towards the right direction. In this function, the motors on one side rotate backward while the motors on the other side rotate forward. Specifically, motor1 and motor2 move backward, while motor3 and motor4 move forward, all at maximum speed (255). This opposite rotation of wheels causes the car to rotate or turn right smoothly. This helps the car change direction effectively when a right command is received from the Bluetooth device.

## 5.7 Stop

```
void Stop()
{
    motor1.setSpeed(0);
    motor1.run(RELEASE);
    motor2.setSpeed(0);
    motor2.run(RELEASE);
    motor3.setSpeed(0);
    motor3.run(RELEASE);
    motor4.setSpeed(0);
    motor4.run(RELEASE);
}
```

This function is used to stop the movement of the car completely. It sets the speed of all four motors to 0, which means no power is given to the motors. After that, each motor is set to RELEASE mode, which stops their rotation immediately. As a result, all wheels stop moving, and the car comes to a halt. This function ensures safe and controlled stopping whenever a stop command is received from the Bluetooth device.

## 5 Theoretical Background

---

### 5.8 Control System Concept :

The working of a Bluetooth controlled car can be understood using basic control system concepts. The system mainly has three parts: input (mobile commands), processing (Arduino), and output (motor movement). These parts work together to control the direction and speed of the car.

The input is given through a mobile phone using Bluetooth, where commands like forward, backward, left, and right are sent. The Arduino acts as the control unit, which reads these commands and decides how the car should move. This is similar to a control system where input signals are processed to produce the desired output.

The output stage consists of the motor driver and DC motors, which convert the Arduino's signals into actual movement of the car. By changing the commands, the user can control the motion of the car in real time

### 5.9 Control Logic Design :

In this project, the control of the car is based on simple programming logic instead of complex mathematical filters. The Arduino reads commands from the Bluetooth module and decides how the motors should behave. This decision-making process acts like a control system where inputs are converted into proper outputs.

The system uses a combination of conditions (like if-else or switch-case) to control direction and speed. This ensures that the car responds correctly to each command such as forward, backward, left, and right. The logic is designed in such a way that the car moves smoothly without sudden or unwanted changes.

The performance of the system depends on how well the code is written. Proper timing and command handling ensure that the car reacts quickly and remains stable during operation. If needed, speed control using PWM can be added to improve smoothness.

## 5.10 Lock Range, Capture Range, and Pull-in Range

| Parameter         | Definition                                            | Value in This Design                 |
|-------------------|-------------------------------------------------------|--------------------------------------|
| Control Range     | Maximum distance over which the car can be controlled | wirelessly without losing connection |
| Response Time     | Time taken by the system to respond to user input     | command                              |
| Motor Speed Range | Range of speeds achievable by DC motors using driver  | control                              |
| Power Consumption | Total power consumed by system during operation       | ~7.4V battery supply (2×18650 cells) |

Table 4:Parameter and Definations

### Signal Delay and Response

In this Bluetooth controlled car project, instead of phase noise and jitter, the main factors affecting performance are small delays in communication and motor response. These delays mainly come from: (1) Bluetooth communication delay between mobile and Arduino, (2) processing time inside the Arduino program, and (3) mechanical response time of the motors.

The Bluetooth module may introduce a slight delay while sending commands, and the Arduino takes a small amount of time to read and process these signals. Additionally, motors do not start or stop instantly due to physical inertia, which creates a small response delay.

Because of these factors, the car may not react perfectly instantly to commands, but the delay is very small and usually not noticeable during normal operation. These limitations do not affect the basic working of the system and the car still performs smoothly and reliably for learning and demonstration purposes.

# 1. Observations

## 1.1 Test Setup

The function generator was set to produce a 1 kHz square wave (0–5 V, 50% duty cycle) on its output channel. The output was connected directly to Arduino Pin 2 via a BNC-to-jumper adaptor. Pin 9 of the Arduino was connected to Channel 2 of the DSO. The Arduino was powered and programmed via USB, and the Serial Plotter was opened at 115200 baud.

## 1.2 Oscilloscope Measurements

| Parameter            | Reference (Pin 2) | Output Pre-Lock (Pin 9) | Output Post-Lock (Pin 9) |
|----------------------|-------------------|-------------------------|--------------------------|
| Signal Type          | Square wave       | Square wave             | Square wave              |
| Frequency            | 1000 Hz           | ~1000 Hz (drifting)     | 1000 Hz (stable)         |
| Amplitude (Vpp)      | 5 V (from gen)    | 5 V (Arduino GPIO)      | 5 V (Arduino GPIO)       |
| Phase Error (Serial) | N/A               | Large, oscillating      | → 0 $\mu$ s (locked)     |
| Lock Time            | —                 | —                       | ~300 ms                  |
| Supply Voltage       | —                 | 5 V (USB)               | 5 V (USB)                |

Table 5: Measured parameters from oscilloscope and Serial Plotter

## 1.3 Serial Plotter Observation

On opening the Serial Plotter immediately after power-on, the currentPhaseError trace shows a large initial deviation (typically  $\pm 500$  to  $\pm 5000$   $\mu$ s depending on the starting frequency offset). Over approximately 20–60 plot updates (1–3 seconds), the error decays and converges to a small residual value near 0  $\mu$ s. This convergence behaviour is the visual signature of PLL lock acquisition and represents the damped second-order step response of the control loop.

Changing the function generator frequency by  $\pm 200$  Hz instantaneously causes a new transient in the error signal, after which the loop re-acquires lock within ~100–200 ms — demonstrating dynamic frequency tracking.



## 2. Explanation of Results

---

- **Successful Movement Control:**

The car responds correctly to all commands like forward, backward, left, right, and stop. This shows that the Bluetooth communication and control system are working properly.

- **Quick Response Time:**

When a button is pressed on the mobile app, the car reacts almost instantly. This confirms that the system works in real time with very small delay.

- **Stable Connection:**

The Bluetooth connection remains stable within a certain range (around 5–10 meters). The car follows commands continuously without interruption as long as it is within range.

- **Smooth Direction Control:**

The motor driver properly controls both motors, allowing smooth turning and direction changes. This shows that the signals from Arduino to the motor driver are correct.

- **Power Supply Effect:**

If the battery voltage is low, the motors run slowly or may not respond properly. This shows that a proper and stable power supply is very important for good performance.

- **PWM Speed Control (if used):**

When PWM is applied, the speed of the car can be increased or decreased smoothly. This helps in better control compared to simple ON/OFF movement.

- **System Reliability:**

The system works continuously without major errors, proving that the design is reliable and suitable for basic robotics applications.

- **Limitations Observed:**

There may be slight delays or signal loss if obstacles block the Bluetooth signal or if the distance becomes too large. Also, uneven surfaces can affect the movement of the car.

## 3. Advantages

---

### 3.1 Implementation Advantages

- **No Complex Components Required:**

The project does not need complicated electronic components like capacitors, ICs, or advanced circuits. It mainly uses simple parts like Arduino, Bluetooth module, motor driver, and motors, making it easy to build.

- **Easy to Modify and Upgrade:**

The system is fully programmable. You can easily change the code to add new features like speed control, obstacle avoidance, or voice control without changing hardware.

- **Wireless Control:**

The car is controlled using Bluetooth, so no wires are needed between the user and the car. This makes it more flexible and user-friendly.

**Real-Time Operation:**

The car responds instantly to commands from the mobile app, allowing smooth and real-time control of movement.

- **Low Cost Project:**

The total cost of the project is very low (around ₹800–₹1500), making it affordable for students and beginners.

- **Easy to Understand:**

This project helps students clearly understand basic concepts like Arduino programming, motor control, and wireless communication.

- **Portable and Compact:**

The car is small and can be easily carried anywhere, making it suitable for demonstrations and learning purposes.

### 3.2 Performance Advantages :

- **Consistent Performance:**

The behavior of the car remains the same every time it runs because it is controlled by a digital program. Unlike analog systems, there is no change due to temperature or component variation, so performance is reliable.

- **Easy Integration with Other Systems:**

Since the car is controlled by a microcontroller (Arduino), it can be easily connected with other modules like sensors (ultrasonic, IR), displays, or even IoT systems. This makes it very flexible for future upgrades.

- **Adjustable Control in Software:**

All control settings (like speed, direction behavior, delay, etc.) can be changed in the program. There is no need to change hardware connections, making testing and modification very easy.

## 4. Limitations

---

### 4.1 Limitation

- **Limited Control Range:**

The Bluetooth works only within a short distance (around 5–10 meters). If the distance increases or obstacles come in between, the signal may become weak or disconnect.

- **Discrete Movement Control:**

The car mainly follows fixed commands like forward, backward, left, and right. It does not move with perfect smooth precision unless advanced programming (like fine PWM tuning) is added.

- **Signal Delay or Noise:**

Sometimes small delays can happen due to Bluetooth communication or processing time. This may slightly affect real-time response, especially when sending rapid commands

### 4.2 System Stability and Control Issues

- **Unstable Movement at High Speed:**

If the motor speed is too high, the car may become difficult to control or may not respond properly to direction changes.

- **Dependence on Battery Power:**

If the battery voltage drops, the motors may slow down or stop working, causing unstable performance.

- **No Automatic Safety Control:**

If the signal is lost, the car may continue its last movement. Extra logic (like auto-stop) is needed for a safer system.

### 4.3 Output and Performance Limitations

- **Simple Motor Output:**

The car uses DC motors which provide basic movement but not very precise control like servo or stepper motors.

- **No Feedback System:**

There is no sensor to check position or speed, so the system works only on given commands (open-loop system).

- **Load Handling Limitation:**

The motor driver and Arduino cannot handle very heavy loads. If weight increases, performance decreases.

## 5. Applications

---

### 5.1 Scientific and Research Applications

- **Robotics Projects:**

Bluetooth controlled cars are widely used in robotics for learning and experiments. Students use them to understand basic concepts of control systems, programming, and automation.

- **Remote-Control Vehicles:**

This system can be used to build wireless vehicles that can be controlled from a mobile phone instead of traditional remotes.

- **Home Automation Systems:**

The same concept of Bluetooth control can be applied to control home devices like lights, fans, and appliances using a smartphone.

- **Industrial Applications:**

In industries, similar wireless control systems are used to operate machines or robots in areas where manual control is difficult or unsafe.

- **Surveillance and Security:**

Bluetooth-controlled robots can be used for monitoring small areas or indoor surveillance where direct human access is limited.

- **Educational Purpose:**

This project is very useful for students to learn Arduino, embedded systems, and wireless communication in a practical way.

• .

### 5.2 Consumer and Industrial Applications

- **Mobile Controlled Vehicles:**

Bluetooth controlled cars are used to create wireless vehicles that can be operated using a smartphone. This replaces traditional remote controls and makes control more flexible and modern.

- **Smart Home Devices:**

The same Bluetooth system can be used in home automation to control lights, fans, doors, and other appliances directly from a mobile phone.

- **Industrial Robot Control:**

In industries, similar systems are used to control small robots or machines wirelessly in areas where manual operation is difficult or unsafe.

- **Automated Transport Systems:**

Bluetooth-controlled vehicles can be used for small-scale material handling in factories, such as moving objects from one place to another.

- **Surveillance Robots:**

These cars can be modified with cameras and sensors for security purposes, allowing remote monitoring of indoor areas.

- **Motor Speed and Direction Control:**

The motor driver used in this project demonstrates how industries control motor speed and direction in machines and automation systems.

- **Assistive Technology:**

Bluetooth-controlled systems can be used in devices like wheelchairs or assistive robots to help physically challenged people move easily using a mobile phone.

- **Smart Energy and Automation Systems:**

Wireless control systems like this are used in smart environments to manage devices efficiently, reducing manual effort and improving automation.

## 5.3 Educational Applications

- **Useful for Student Learning:**

This project is very helpful for students in electronics, robotics, and engineering courses. It helps them understand basic concepts like Arduino programming, motor control, and wireless communication in a practical way.

- **Practical Understanding of Theory:**

Students can connect theory with real hardware. Instead of only studying concepts, they can see how commands from a mobile actually control the movement of a car.

- **Hands-on Experience:**

This project gives hands-on experience in building circuits, connecting components, and writing code, which improves technical skills.

- **Easy Visualization of Working:**

The movement of the car (forward, backward, turning) clearly shows how the system responds to inputs, making it easier to understand compared to theoretical learning.

## 6. Literature Review

Bluetooth controlled car technology has been widely studied in both academic and practical fields as a basic application of embedded systems and wireless communication. With the development of microcontrollers like Arduino, students and researchers have been able to design simple robotic systems that demonstrate real-time control and automation.

Many researchers and developers have explained the use of Arduino in robotics, showing how it can act as the main controller to process inputs and control outputs. The use of Bluetooth modules like HC-05 has become very popular because it provides an easy and low-cost way to achieve wireless communication between a mobile phone and a robotic system.

Recent developments in embedded systems have made it possible to design fully programmable robots, where all control logic is implemented in software. This allows easy modification, testing, and improvement without changing hardware components. The concept used in this project is similar to modern smart systems where devices are controlled wirelessly using mobile applications.

Educational resources and Arduino documentation also provide detailed guidance on motor drivers like L298N, which are essential for controlling motor direction and speed. These studies help in understanding how low-power control signals from a microcontroller can be used to operate high-power devices like motors.

Overall, this project is based on well-established concepts of wireless communication, embedded programming, and motor control, making it a strong practical example of modern robotics and automation systems.

| Author(s)                     | Year | Key Contribution                                                                                     |
|-------------------------------|------|------------------------------------------------------------------------------------------------------|
| Arduino.cc                    | 2023 | Official Arduino documentation — microcontroller programming, GPIO control, and serial communication |
| Texas Instruments (TI)        | 2015 | L293D Motor Driver datasheet — H-bridge operation and motor control principles                       |
| HC-05 Module Documentation    | 2018 | Bluetooth communication using UART — wireless data transmission between mobile and Arduino           |
| GitHub / Open Source Projects | 2020 | Bluetooth RC car implementations — command-based control (F, B, L, R) logic                          |
| Android Developers            | 2022 | Bluetooth app development — mobile interface for sending control commands                            |

## 7. Practical Implications

---

### 7.1 Embedded Systems Design

This project shows that a complete control system can be built using a simple and low-cost microcontroller like Arduino instead of using complex and expensive hardware circuits. In earlier systems, special electronic components were required to control devices, but now the same work can be done using software (programming) inside the microcontroller.

In the Bluetooth controlled car, all the control logic—like receiving signals, processing commands, and controlling motors—is done through Arduino programming. This reduces the need for extra components, making the system simpler, cheaper, and easier to design.

This approach is very useful in modern embedded systems such as IoT devices, smart robots, and automation systems. Designers can build advanced systems using only a microcontroller and software, which reduces circuit complexity, saves space, and makes the system more flexible and easy to upgrade.

### 7.2 Control Systems Education

This project is very useful for understanding control systems in an easy and practical way. By using a Bluetooth controlled car, students can observe how different inputs (like forward, backward, left, and right commands) affect the movement of the system in real time. It helps in understanding how control actions work, such as how changing speed or direction affects the response of the car. Instead of only learning theory, students can clearly see how the system reacts to commands, making difficult concepts easier to understand through hands-on experience and visualization.

### 7.3 Scalability

The design of this project can be easily improved by using faster and more advanced microcontrollers. For example, boards like STM32 or RP2040 can process data much faster, allowing better performance, quicker response, and smoother control of the car. The same control logic used in this project can be applied to more advanced systems with higher speed and better accuracy. For very high-speed or complex applications, special hardware modules may be used, but the basic working principle remains the same. This shows that the Bluetooth controlled car project is scalable and can be upgraded for more advanced robotics and automation systems in the future.

### 7.4 Industrial and Safety Considerations

In real-world and safety-critical applications, systems must be more reliable than this basic project. Extra features like protection against errors and system failure are required. For example, safety checks can stop the car if signals are lost or invalid. Timers can be used to detect if the system stops working properly. It is also important to ensure stable performance even with changes in battery or environment. These improvements make the system safer and suitable for advanced industrial use.

## 8. Advantages and Limitations — Summary

---

### 8.1 Advantages

- **No Complex Components Required:**

The project uses simple components like Arduino, Bluetooth module, and motor driver, so it is easy to build and understand.

- **Fully Programmable System:**

All functions like movement and speed control are defined in software, so changes can be made easily without modifying hardware.

- **Real-Time Control:**

The car responds instantly to commands from the mobile phone, providing smooth and quick operation.

- **Stable Performance:**

Since the system is digital, its performance does not change much with temperature or environmental conditions.

- **Low Cost:**

The project is affordable and suitable for students, making it ideal for learning and experiments.

- **Easy to Upgrade:**

It can be improved by adding sensors like ultrasonic, IR, or voice control for advanced features.

- **Reliable Operation:**

The system works continuously and gives consistent results, making it suitable for basic robotics applications.



## 8.2 Limitations

- **Limited Range:**

The Bluetooth connection works only within a short distance (about 5–10 meters). Beyond this range, control may be lost.

- **Simple Control Only:**

The car follows basic commands like forward, backward, left, and right. It does not have advanced control like automatic navigation unless extra sensors are added.

- **Small Delay in Response:**

There may be a slight delay due to Bluetooth communication and processing inside the Arduino.

- **Single Control System:**

The car is controlled by only one device at a time and cannot handle multiple users or complex control systems.

- **No Safety Features:**

If the Bluetooth connection is lost, the car may continue its last action. Extra programming is needed for safety (like auto-stop).

- **Battery Dependency:**

Performance depends on battery power. Low battery can reduce speed or stop the car.

- **No Feedback System:**

There are no sensors to check position or speed, so the system cannot correct itself automatically.

## 9. Conclusion

---

In this project, a Bluetooth controlled car was successfully designed and implemented using an Arduino UNO microcontroller. The system uses a Bluetooth module (HC-05) to receive commands from a mobile phone, and these commands are processed by the Arduino to control the movement of the car. A motor driver (L298N) is used to drive the DC motors, allowing the car to move forward, backward, left, and right based on user input.

The working of the system was clearly observed during testing. The car responds to commands in real time, showing smooth and accurate control. The wireless communication through Bluetooth works effectively within a limited range, and the overall system performs reliably under normal conditions.

Through this project, the functions of different components such as the Arduino, Bluetooth module, motor driver, and motors were understood practically. It also helped in connecting theoretical concepts of embedded systems and wireless communication with real-world implementation.

Overall, this project demonstrates the importance of simple and cost-effective automation systems. It shows how basic components and programming can be used to build useful robotic applications, and it provides a strong foundation for developing more advanced systems in robotics and automation.

## 10. References

---

1. Gardner, F. M. (1979). Phaselock Techniques (2nd ed.). John Wiley & Sons.
2. Best, R. E. (2007). Phase-Locked Loops: Design, Simulation, and Applications (6th ed.). McGraw-Hill.
3. Razavi, B. (2003). Design of Analog CMOS Integrated Circuits. McGraw-Hill.
4. Viterbi, A. J. (1966). Principles of Coherent Communication. McGraw-Hill.
5. Sedra, A. S., & Smith, K. C. (2014). Microelectronic Circuits (7th ed.). Oxford University Press.
6. Atmel Corporation. (2006). AVR135: Using Timer/Counter in CTC Mode. Application Note.
7. Perrott, M. H. (2012). All-Digital Phase-Locked Loops: Concepts, Design and Applications. IEEE Solid-State Circuits Magazine, 4(2), 8–21.
8. Staszewski, R. B., & Balsara, P. T. (2011). All-Digital Frequency Synthesizer in Deep-Submicron CMOS. Wiley-IEEE Press.
9. Horowitz, P., & Hill, W. (2015). The Art of Electronics (3rd ed.). Cambridge University Press.
10. Arduino LLC. (2024). Arduino UNO R3 Datasheet and Reference Manual. arduino.cc.
11. Microchip Technology. (2015). ATmega328P Complete Datasheet (DS40002061B). microchip.com.
12. Electronics Circuits Lab Manual — Department of E&C, Institute of Technology, Nirma University, 2025–26.